

# Spring DTO (Data Transfer Objects)



**Spring Boot**

DTO

# Plan du Cours

- Introduction
- Schéma "Sandwich" des DTOs
- Distinction DTO avec les entités
- Pourquoi utiliser les DTOs
- Les approches mapping : Le mapping manuel + le mapping automatique

# Introduction

Dans le développement d'applications Spring Boot, les entités JPA sont utilisées pour représenter les tables de la base de données et gérer la persistance des données.

Toutefois, dans le cadre des échanges entre l'application back-end et les interfaces externes (API REST, front-end, services tiers), il est déconseillé d'exposer directement ces entités.

# Introduction

- **l'exposition directe des entités:**

L'utilisation directe des entités dans les contrôleurs présente plusieurs inconvénients :

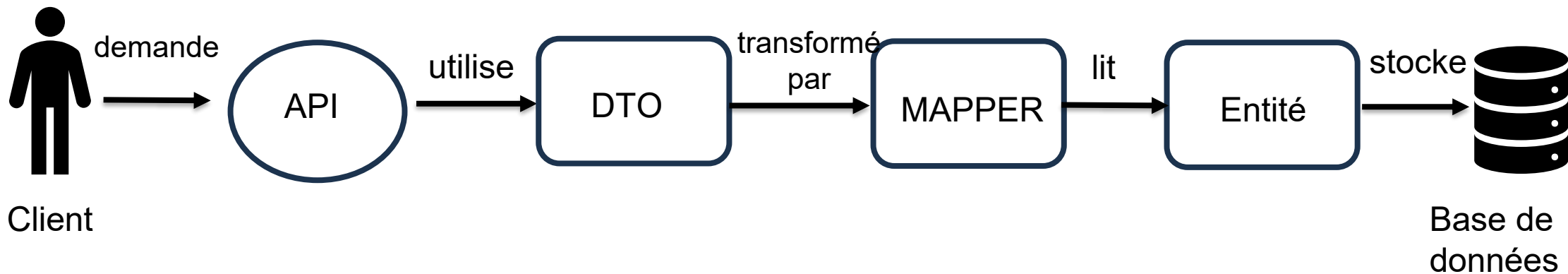
1. Risque d'exposer des **données sensibles** ou inutiles.
2. Présence de **relations complexes** pouvant entraîner des problèmes de sérialisation (ex. : boucles infinies).
3. Forte dépendance entre le modèle de persistance et l'interface exposée (front-end, API), rendant le système **fragile** lors des évolutions.
4. Besoin fréquent de structurer les données de sortie de manière différente de la structure de base.

# Introduction

Pour éviter les inconvénients de l'exposition directe des entités, le **DTO** (*Data Transfer Object*) est un objet **spécialement conçu pour le transfert de données** entre la couche back-end et les autres couches (REST, interface utilisateur, services externes...).

Il contient **uniquement les champs nécessaires à l'opération ciblée** (exemple : afficher un projet, créer une entreprise, etc.).

# Schéma "Sandwich" des DTOs



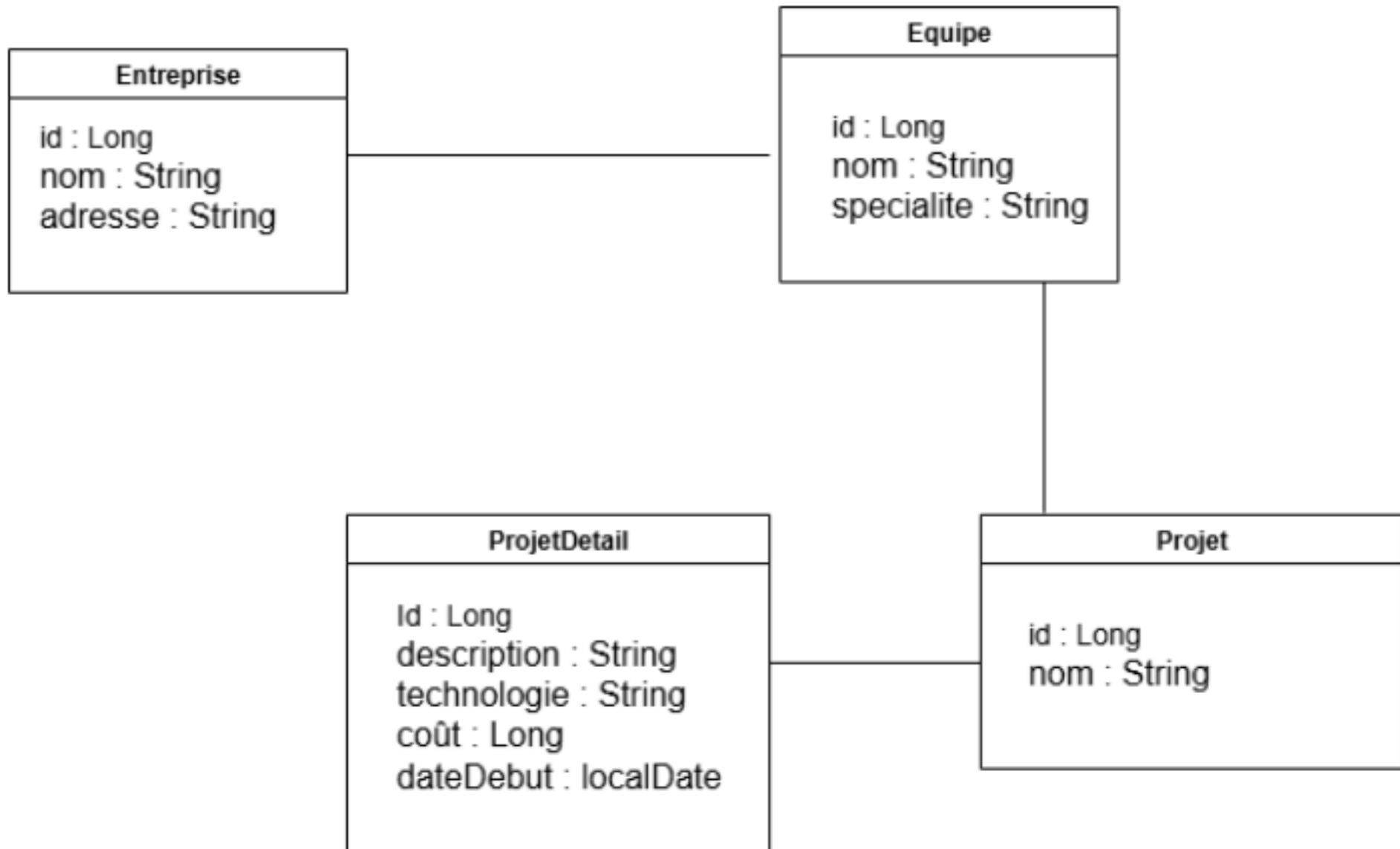
1. **Client** : Demande des données ("Je veux un sandwich").
2. **API** : Prend la commande ("Un jambon-beurre sans cornichons").
3. **DTO** : Le menu simplifié ("Sandwich Jambon").
4. **MAPPER** : Le cuisinier qui enlève les cornichons (champs sensibles).
5. **Entité** : Tous les ingrédients en cuisine (base de données).

# Distinction avec les entités

- **Entité** : est une classe Java annotée avec `@Entity` Elle représente une **table dans la base de données**. Chaque attribut correspond à une **colonne**, et les relations représentent les **liens entre les tables**.
- **DTO** : la version allégée pour les échanges

Un **DTO (Data Transfer Object)** est une classe **non liée à la base de données**. Elle sert à transporter **seulement les données nécessaires** entre le backend et le frontend (ou entre deux services).

# Diagramme de classes





# Distinction avec les entités

## Exemple :

- Entité **ProjetDetail** représente les détails d'un projet donné. Elle contient les **attributs complets** nécessaires à la gestion technique et administrative d'un projet : **id, description, technologie, cout, dateDebut**.
- Le DTO **projetDetailDTO** est utilisé pour **transporter uniquement les informations nécessaires** vers l'interface utilisateur, sans exposer de logique métier ou d'informations sensibles. Les attributs typiques sont : **id, description, technologie**. Ces champs sont **suffisants** pour afficher une **liste de projets** avec leurs descriptions et technologies dans une vue utilisateur.

# Pourquoi utiliser les DTO?

L'utilisation des DTOs dans une application Spring Boot offre plusieurs avantages significatifs :

1. **Séparation des préoccupations** : Les entités représentent le modèle de données persistant, les DTO représentent le modèle exposé via l'API
2. **Sécurité** : Évite d'exposer des données sensibles présentes dans les entités
3. **Performance** : Permet de ne transférer que les données nécessaires
4. **Flexibilité** : Permet d'avoir une structure de données différente entre la base et l'API

# Approches Mapping : Le mapping manuel Entité ↔ DTO

La conversion manuelle consiste à transformer les entités en objets DTO à **la main**, sans utiliser de bibliothèque externe.

Les avantages et les Inconvénients de ce type de conversion :

| Avantages                                          | Inconvénients                                   |
|----------------------------------------------------|-------------------------------------------------|
| Contrôle total sur le processus de transformation. | Répétitif si plusieurs entités sont concernées. |
| Facile à déboguer et à personnaliser.              | Peu maintenable à grande échelle.               |
| Ne dépend d'aucun outil ou librairie externe.      | Risque d'oubli de champs lors de la conversion. |

# TP : Implémenter le mapping manuel Entité ↔ DTO

Transformer **ProjetDetail** en **DTO** en masquant le cout et en simplifiant la date.

## Entité ProjetDetail

```
@Entity
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class ProjetDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String description;
    private String technologie;
    private Long cout; //champ à cacher
    private LocalDate dateDebut; //champ à reformater
}
```

# TP : Implémenter le mapping manuel Entité ↔ DTO

## DTO ProjetDetailDTO

- Pas de id ni cout (masqués pour l'API)
- dateDebut : Chaîne formatée au lieu de LocalDate

@Getter 6 usages

@Setter

@AllArgsConstructor

@NoArgsConstructor

```
public class ProjetDetailsDTO {  
    private String description;  
    private String technologie;  
    private String dateDebut; // Exemple: "19/07/2025" au lieu de LocalDate  
}
```

# TP : Implémenter le mapping manuel Entité ↔ DTO

## Mapper manuel (Service)

```
public ProjetDetailsDTO getDetailsProjet(Long id) { no usages
    ProjetDetail detail = projetDetailRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Détails du projet non trouvés"));

    // Conversion Entité → DTO
    return convertToDto(detail);
}
```

```
private ProjetDetailsDTO convertToDto(ProjetDetail detail) { 1 usage
    ProjetDetailsDTO dto = new ProjetDetailsDTO();
    dto.setDescription(detail.getDescription());
    dto.setTechnologie(detail.getTechnologie());

    // Changement de la date
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    dto.setDateDebut(detail.getDateDebut().format(formatter));

    return dto;
}
```

} Le mapping manuel copie seulement les champs nécessaires

# TP : Implémenter le mapping manuel Entité ↔ DTO

**Le contrôleur :** Utilise le DTO pour retourner la réponse API.

```
@GetMapping("/{id}/details")
public ProjetDetailsDTO getDetails(@PathVariable Long id) {
    return projetDetailService.getDetailsProjet(id);
}
```

## Le résultat

| Code | Details                                                                                                                                                                     |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div>Response body<pre>{   "description": "Une petite description",   "technologie": "Angular, SpringBoot, Flask, .NET, Symfony",   "dateDebut": "15/07/2025" }</pre></div> |

# Approches Mapping : Le mapping automatique

## Entité ↔ DTO avec MapStruct

L'implémentation manuelle des DTOs, bien que fonctionnelle, présente plusieurs **limitations** en termes de productivité et de maintenance.

**MapStruct** offre une alternative robuste pour la conversion entre entités et DTOs, avec ces avantages principaux :

- Réduction significative du code répétitif lié aux opérations de mapping.
- Génération de code à la compilation garantissant des performances Optimales.
- Flexibilité pour gérer les cas de conversion complexes.
- Meilleure maintenabilité du code grâce à une séparation claire des responsabilités.



# TP : Implémenter le mapping automatique Entité ↔ DTO avec MapStruct

**Dépendances:** il faut ajouter les bonnes dépendances dans pom.xml pour utiliser MapStruct.

```
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct</artifactId>
    <version>1.5.5.Final</version>
</dependency>
<dependency>
    <groupId>org.mapstruct</groupId>
    <artifactId>mapstruct-processor</artifactId>
    <version>1.5.5.Final</version>
    <scope>provided</scope>
</dependency>
```

# TP : Implémenter le mapping automatique Entité ↔ DTO avec MapStruct

## Entité Projet

```
@Entity
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class Projet {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nom;
}
```

# TP : Implémenter le mapping automatique Entité ↔ DTO avec MapStruct

## DTO ProjetDTO

Contient uniquement les données à exposer.

On renomme nom à libelleProjet pour voir que le DTO peut avoir une structure différente.

```
@Getter 4 usages
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class ProjetDTO {
    private String libelleProjet; // 'nom' renommé pour l'API
}
```

# TP : Implémenter le mapping manuel Entité ↔ DTO

**Mapper MapStruct** : Définit comment convertir `Projet` → `ProjetDTO`  
On n'a pas besoin d'écrire manuellement le code qui effectue la conversion entre l'entité `Projet` et le DTO `ProjetDTO`.

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;
import tn.esprit.dto.DTOs.ProjetDTO;
import tn.esprit.dto.entities.Projet;

@Mapper(componentModel = "spring") no usages
public interface ProjetMapper {

    @Mapping(target = "libelleProjet", source = "nom") // Renommage no usages
    ProjetDTO toDto(Projet projet);
    // MapStruct génère automatiquement l'implémentation !
}
```

# TP : Implémenter le mapping manuel Entité ↔ DTO

L'interface **ProjetMapper** est annotée avec **@Mapper**. MapStruct va générer à la compilation une classe concrète avec la méthode `toDto`.

```
@Service no usages
@AllArgsConstructor
public class ProjetService {

    private final ProjetRepository projetRepository;
    private final ProjetMapper mapper;

    public ProjetDTO getProjet(Long id) { no usages
        Projet projet = projetRepository.findById(id).orElseThrow( exceptionSupplier: null);
        return mapper.toDto(projet); // Conversion en 1 ligne
    }
}
```

# TP : Implémenter le mapping manuel Entité ↔ DTO

L'interface **ProjetMapper** qui est écrite est une interface déclarative. Grâce à l'annotation **@Mapper**, MapStruct comprend qu'on veut créer un mapper (convertisseur) entre deux objets.

**MapStruct** va automatiquement générer une classe concrète (pendant la compilation) avec une implémentation de la méthode `toDto`.

💡 Gain de temps, moins d'erreurs, code plus lisible et maintenable.

# TP : Implémenter le mapping manuel Entité ↔ DTO

## Contrôleur

```
@GetMapping("/{id}")
public ProjetDTO getProjet(@PathVariable Long id) {
    return projetService.getProjet(id);
}
```

## Résultat

| Code | Details                                                              |
|------|----------------------------------------------------------------------|
| 200  | <div>Response body</div> <pre>{   "libelleProjet": "LIBELLE" }</pre> |

# Utilité DTO

La valeur ajoutée des DTO est incontestablement ressenti dans le cas où nous avons des jointures entre les tables.

L'utilisation des DTO réduit considérablement le nombre d'attributs exposés ce qui nous permet d'avoir des objets moins complexes et un temps de réponse réduit.



# Utilité DTO

Donnons un exemple dans notre équipe de cas où on veut créer un service récupérant la date d'une commande, le total de la commande, le libelle du menu associé à cette commande ainsi que l'identifiant du client.

En utilisant les dtos, on peut réduire le nombre d'attributs à exposer de 14 attributs (associés aux 3 entités) à 4 seulement.

Les 4 champs du DTO permettent d'avoir la réponse métier optimale à l'utilisateur final.

# Spring DTO



*Utiliser les DTOs, c'est rendre  
votre application plus propre,  
plus sûre et plus évolutive.*

# **Spring DTO (Data Transfer Objects)**

Si vous avez des questions, n'hésitez pas à nous contacter :

**Département Informatique  
UP ASI**

Bureau E204